

Writing a Test-suite Regression Environment (TRE)

coreBuilder Application Note

Paul Furnanz, Synopsys Design Reuse Group

October 2000

Abstract

This application note describes how to incorporate an already written core regression environment into a packaged coreKit. This note describes the requirements of the already existing regression environment, the commands that coreBuilder provides to include that environment in the coreKit, and the commands provided to launch the regression environment within coreConsultant.

Introduction

The coreBuilder tools support configurable IP. Since this IP is configurable, its verification environment should also be configurable. This application note describes how to make a verification environment configurable by coreConsultant. This document also describes the support in coreBuilder for running external tools and reporting the results of external tool runs to users.

This application note assumes the reader has read the following application notes:

- [Using the Active Bill of Materials](#)
- [Parameters and the Automatic Dialog Builder](#)
- [Customizing the coreBuilder and coreConsultant Activity Flow](#)

Test-suite Requirements

All test-suites brought into the coreBuilder tools should run in batch mode, and be self-checking. The coreBuilder tools do not know how to run or set up any simulators, so it is the responsibility of the writer of the test-suite to deal with this.

All verification environments are composed of one or more configurable filegroups under a filegroup that has the DeliverableType set to “TRE”.

Organization of the Verification Environment

There are two ways to organize the verification environment for the end user. The first method is called *monolithic*, and the second *parallel*.

A Monolithic Environment

A monolithic verification environment consists of only one activity presented to the user of the coreKit. This type of regression environment is the easiest to set up. To create a verification environment with this organization create one configurable filegroup that has all the verification-related files. This filegroup should have the DeliverableType set to “TRE”.

There should be parameters on this filegroup to select the simulator (if multiple simulators are supported), one to select the path to that simulator, and then boolean parameters so the user can select which tests to run. The GroupName or ParameterInfo attributes should be used to group the various tests (boolean parameters) into different test-suites.

A Parallel Environment

A parallel verification environment consists of multiple activities. There is one activity to set up the verification environment (select a simulator, etc.), and then one activity for each test-suite. This organization is nice, because the different test-suites can run in parallel.

This type of environment uses a hierarchy of filegroups. The top-most group should have its DeliverableType set to TRE. This group should contain the files and parameters needed to configure the entire verification environment. Each test-suite should be a filegroup that is a child of this TRE group. These filegroups should contain boolean parameters so the user can select which tests to run.

Running the Simulator within coreConsultant

The coreBuilder tools provide a generic method for launching external tools, and then reporting the results of those tool runs to the user. The command `launch_activity_subproc` runs external tools for activities that support sub-processes. To give an activity

support for sub-processes, the command `define_activity_subproc_params` is used.

Defining Sub-process parameters

The `define_activity_subproc_params` command defines parameters on an activity or configurable filegroup, so that the coreBuilder tools can run an external tool. This command defines a set of parameters that are intended for user control and another that the writer of the test-suite should customize.

The parameters intended for user control are in the following table:

Name	Valid Values	Description
ScriptsOnly	0, 1	Don't actually run the command, just create a run.scr file so that it can be run later.
RunStyle	local, lsf, remote	If ScriptsOnly is 0, then this selects how to run the command. The command can be run locally, through LSF, and via the remote shell.
RemoteHost	<any host>	If the RunStyle is remote, then this is the name of the machine to run the command on.
LsfOptions	<any>	If the RunStyle is lsf, this parameter has any special options to pass to the bsub (or csub if available) command.
EmailAddress	<any>	If this parameter is not set to "", then email is sent when the job is completed or killed to that email address (default is the current user name).

The parameters intended for customization are in the following table. These parameters are not Visible, and will not show up in the dialog presented to users.

Name	Description
WorkDirectory	Set this parameter to the directory where the scripts and guard files go. The command will also run in this directory. No two activities should have the same value for this parameter. The value for this parameter should be a relative path name from the user's workspace.
ResultsFile	Set this parameter to the location of the result file for the external command. The path to this should be relative to the WorkDirectory,. The preferred format of this file is HTML.
MailFile	If this parameter is set, then the contents of this file are appended to the generic mail message that is sent when the external command completes (if EmailAddress is set).

If your TRE filegroup is called `Verification_Environment`, then to use this feature put the following into the intent file for the “`Verification_Environment`” filegroup:

```
set group Verification_Environment
# Make launch-able
define_activity_subproc_params $group -workdir sim \
    -resultsfile results.html -mailfile .summary
set_filegroup_attribute $group PostPromptCmd \
    {MyCore_LaunchTestSuite %item}
```

In this example, the command `MyCore_LaunchTestSuite` is defined in the same intent file using the `plugin_proc` command. An example implementation of this command is in the next section.

Launching an External Tool

Once an activity or filegroup has the correct settings on the sub-process parameters, the external tool is launched with the `launch_activity_subproc` command. This command accepts the activity and command string to launch. This command string is a bourne shell (`/bin/sh`) script, with multiple commands separated by either semi-colons or new-lines.

To continue the above example, the following code is included in the TCL intent file:

```

plugin_proc MyCore_LaunchTests { act } {
  # First launch the regressions
  set cmd "./runtest.pl testlist 2>&1 > run.log\n"
  # Now parse the output and build the HTML report
  append cmd "./gen_html.pl regress.log > results.html\n"
  # Send some summary information to the user in email.
  append cmd "./gen_mail.sh regress.log > .summary\n"

  # OK, now do it.
  launch_activity_subproc $act $cmd
}

```

Note the use of `plugin_proc` to define the command. This tells `coreConsultant` that the given procedure needs to be available in `coreConsultant`. This is equivalent to defining this procedure in a plugin KB, but is much simpler since all of the code resides directly in the intent file. This command assumes it is being run from `WorkDir` as defined by the `-workdir` option to `define_activity_subproc_params`.

Complete Example

This section describes a complete example of a configurable verification environment. This includes getting the environment loaded into `coreBuilder`, and launching the simulation from within `coreConsultant`.

Commands to Load Environment

This is accomplished through either the GUI, or a batch script. This section gives a batch script that will load the verification environment and its plugin into `coreBuilder`. The next sections describe the files that are loaded by this code.

```

# Get the TRE loaded
create_autoload_filegroup Verification_Environment \
  -patterns {sim/*.v sim/*.pl sim/*.mem sim/testlist sim/*.sh} \
  -install sim
set_filegroup_attribute Verification_Environment DeliverableType TRE
set_filegroup_attribute Verification_Environment Label \
  {Verification Environment}
set_filegroup_attribute Verification_Environment Configurable 1

```

Contents of Certain Files

This section gives the contents of important files. It shows how the files are configured and how the tests are run. The verilog (*.v from above) and the test input files (*.mem from above) are not described here.

File sim/runtest.pl

This Perl file runs the actual simulation. This example only supports the VCS simulator. This file uses text substitution to find the perl path in the coreConsultant installation, and to get the value of the VCS_HOME parameter.

```
# reuse-pragma startSub [format %subText $sysCmd(perl)]
#!%s

# reuse-pragma startSub [ReplaceFormatParam %subText VCS_HOME]
$ENV{VCS_HOME} = "%s";

# First compile the simulator
unlink("simv");
system "$ENV{VCS_HOME}/bin/vcs +nopliopt -PP ../src/risc8_constants.v
../src/risc8_parameters.v ../src/risc8_alu.v ../src/risc8_regb_biu.v
../src/risc8_control.v ../src/risc8.v ./DW01_add.v ./test.v";
die "Cannot create simv executable\n" unless (-x "simv");

# Now run the tests listed
open (LOG, ">regress.log") || die "Can't create log file: $!\n";
select LOG;
while (<>) {
    chomp; next if /^#\//;    next if /^$/;
    print "\nStarting $_ test.....\n";
    print "-----\n";
    unlink("test.mem"); symlink($_, "test.mem"); # Link test.mem
    open (SIM, "./simv|") || die "Can't start simulation: $!\n";
    print while (<SIM>);
}
```

File sim/runtest.tcl

This is the intent file for the runtest.pl described in the previous section. When the Verification_Environment filegroup is analyzed this file is sourced.

```
# Define the look of the dialog, and place it within the flow
set group Verification_Environment
set_filegroup_attribute $group ParameterInfo {{ContainerType tree}}
set_filegroup_attribute $group ActivityGroup Verification
set_filegroup_attribute $group ConfigDependsOnActivities \
    SpecifyConfiguration

# Currently only support VCS...
set p [create_configuration_parameter -group $group VCS_HOME \
```

```

        -type string \
        -default {=if {[catch {getenv VCS_HOME}]} {getenv VCS_HOME}} \
        -label "VCS Installation Path" \
        -description \
            "The setting for the VCS_HOME environment variable" \
    ]
set_attribute $p -attr ValueType -value directory
set_attribute $p -attr GroupName -value Setup

# Define launch parameters.
set p [define_activity_subproc_params $group -workdir sim \
    -resultsfile results.html -mailfile .summary]
set_filegroup_attribute $group PostPromptCmd {MyCore_LaunchTests %item}

# Define test selection parameters.
set testNames { and arith divide flags interrupt
                jmp loadstore logic moves multiply
                or reg }
foreach testName $testNames {
    set p [create_configuration_parameter $testName \
        -type boolean \
        -label $testName \
        -default 1]
    set_attribute $p -attr GroupName -value "Tests"
}

plugin_proc MyCore_LaunchTests { act } {
    # First launch the regressions
    set cmd "./runtest.pl testlist 2>&1 > run.log\n"
    # Now parse the output and build the HTML report
    append cmd "./gen_html.pl regress.log > results.html\n"
    # Send some summary information to the user in email.
    append cmd "./gen_mail.sh regress.log > .summary\n

    # OK, now do it.
    launch_activity_subproc $act $cmd
}

```

File sim/gen_html.pl

This Perl script parses the simulator output and generates a table to summarize the results of the simulation run.

```

# reuse-pragma startSub [format %subText $sysCmd(perl)]
#!%S

# Print a standard header
print <<"EOF";
<HTML>
    <HEAD> <title>Regression Results</title> </HEAD>
    <BODY text="#000000" bgcolor="#FFFFFF">
    <H1>Regression Results</H1>
EOF
;

# Build a table with the results
print <<"EOF";
<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=1>
    <TR>
        <TD> <b>Test Name</b> </TD>
        <TD> <b>Status</b> </TD>

```

```

</TR>
EOF
;

# Parse adding table entries
$found_test = 0;
while (<>) {

    (/^Starting [a-z.]+ test/) && do {
        # Show results of last test
        if ($found_test) {
            print "<TR> <TD>$test</TD> <TD>$status</TD> </TR>\n";
        }
        ($junk, $test, $junk) = split;
        $test =~ s/\.mem//;
        $status = "Ok";           # Assume pass, unless find otherwise
        $found_test = 1;
    };

    /Failed/ && do {
        $status = "<FONT COLOR=red>FAILED</FONT>"
    };
}

# Last test needs to be printed too!
print "<TR> <TD>$test</TD> <TD>$status</TD> </TR>\n";

# Close off the table
print "</TABLE>\n";

```

File sim/gen_mail.sh

This Bourne-Shell script parses the simulator output to generate a summary for the number of failed tests. coreConsultant adds this summary info the a standard mail message when the simulation is complete.

```

#!/bin/sh
f='grep Failed $1 | wc -l'
echo "Failed Tests: $f"

```

File sim/testlist

This file contains the list of tests to run. The file is configured, so that the user can select the list of tests to run. The parameters that allow test selection are defined in the sim/runtest.tcl file. The parameter `conditional_text` command returns "" if the parameter expression is false, and the first argument (%subText in this example) when the parameter expression is true.

```

#List of tests

```

```

# reuse-pragma startSub [parameter_conditional_text %subText @and]
and.mem
# reuse-pragma startSub [parameter_conditional_text %subText @arith]
arith.mem
# reuse-pragma startSub [parameter_conditional_text %subText @divide]
divide.mem
# reuse-pragma startSub [parameter_conditional_text %subText @flags]
flags.mem
# reuse-pragma startSub [parameter_conditional_text %subText @interrupt]
interrupt.mem
# reuse-pragma startSub [parameter_conditional_text %subText @jmp]
jmp.mem
# reuse-pragma startSub [parameter_conditional_text %subText @loadstore]
loadstore.mem
# reuse-pragma startSub [parameter_conditional_text %subText @logic]
logic.mem
# reuse-pragma startSub [parameter_conditional_text %subText @moves]
moves.mem
# reuse-pragma startSub [parameter_conditional_text %subText @multiply]
multiply.mem
# reuse-pragma startSub [parameter_conditional_text %subText @or]
or.mem
# reuse-pragma startSub [parameter_conditional_text %subText @reg]
reg.mem

```